

- ▶ No single model is perfect. Different models make different types of errors.
- ▶ Let's visualize the predictions of two different models:

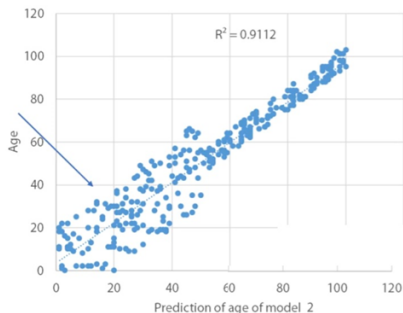
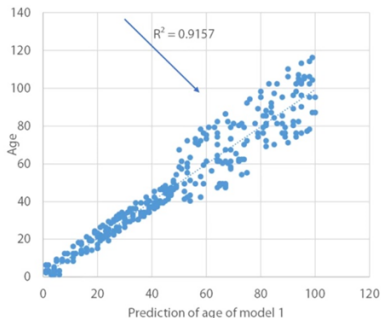


Figure 4: Predictions of Model 1 and Model 2

- Combining predictions of diverse models can reduce errors and improve accuracy. This technique is called **Ensembling**.

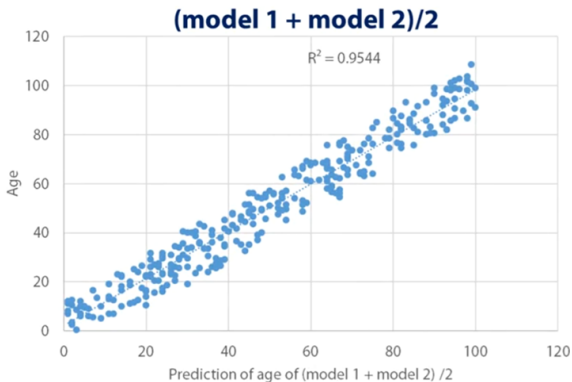


Figure 5: Averaged Predictions: Better Correlation

- ▶ Diverse models are key to effective ensembling. Here are two strategies:
 - **Bagging (Bootstrap Aggregating):** Train multiple models on different subsets of data (e.g., Random Forest model).

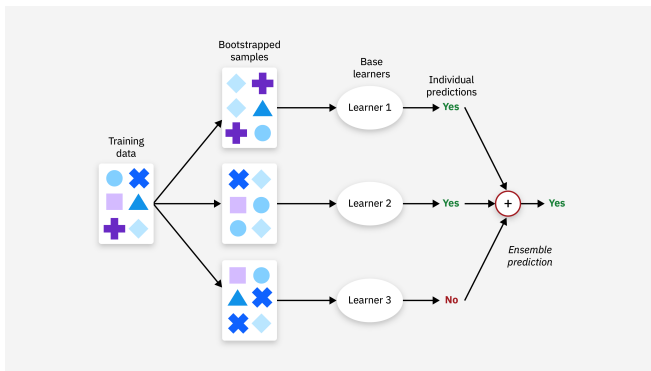


Figure 6: Bagging example

- **Boosting:** Train models sequentially, where each model focuses on the errors of the previous one (e.g., AdaBoost, Gradient Boosting).

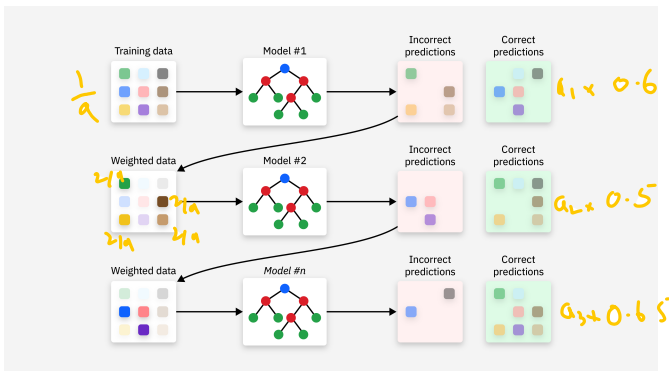


Figure 7: Boosting Example

- ▶ These strategies introduce diversity in models, but how can we combine their predictions?

- ▶ We can combine classifiers predictions using two ways:
 - **Hard Voting:**
 - ▶ Each model votes for a class.
 - ▶ The class with the majority of votes is selected.
 - **Soft Voting:**
 - ▶ Average the predicted probabilities from each model.
 - ▶ The class with the highest average probability is selected.

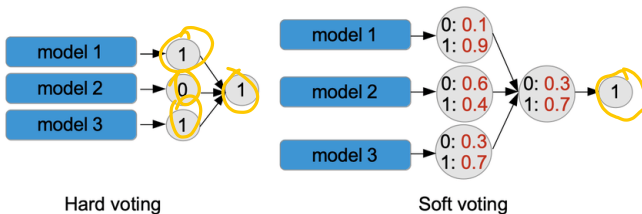


Figure 8: Illustration of Hard and Soft Voting for Classifiers

- ▶ We can combine classifiers predictions using two ways:
 - **Hard Voting:**
 - ▶ Each model votes for a class.
 - ▶ The class with the majority of votes is selected.
 - **Soft Voting:**
 - ▶ Average the predicted probabilities from each model.
 - ▶ The class with the highest average probability is selected.
- ▶ For regressors:
 - Take the average of predictions from all models.

- ▶ Team-work is the best policy.
- ▶ We can train multiple networks for the same task then ensemble to get better results.

- ▶ Let's assume that we have a test dataset with N elements and an ensemble of M models.
- ▶ Also assume that the probability of error of the label for an image on a model in the ensemble is denoted by $p(e)$ and is Independent and Identically Distributed (i.i.d)
- ▶ For an example assume $M = 3$ and $e = 0.01$
- ▶ Then probability of error of label for the max voting ensemble will be

$$p(e) = 1 - (1 - e)^3 - \binom{3}{2}(1 - e)^2e$$

- ▶ For the above example $p(e) = 0.0003$, which is significantly lower than a single model

$$M = 3$$

$$e = 0.1 \quad (\text{accuracy} = 0.9)$$

$$\text{Ensemble error} = \binom{3}{3} 0.1^3 + \binom{3}{2} 0.1^2 0.9$$

$$= 1 \times 0.001 + 3 \times 0.01 \times 0.9$$

$$= 0.001 + 0.027$$

$$= 0.028 \quad (0.972)$$

$$\frac{3!}{3!(3-3)!}$$

$$\frac{1}{0!} = 1$$

$$\frac{3!}{2!(3-2)!}$$

$$\frac{3 \times 2 \times 1}{2 \times 1 \times 1}$$

$$\frac{3}{3}$$

10

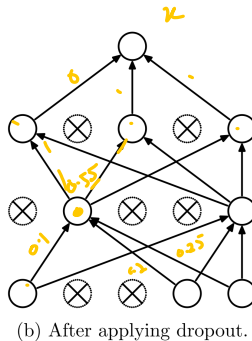
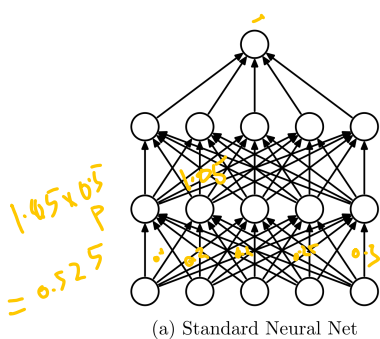
↓

3

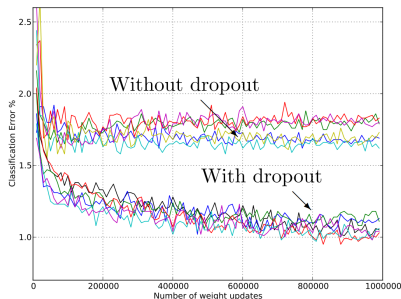
$$\binom{10}{3}$$

$$= \frac{10!}{3! (10-3)!}$$

- ▶ **Dropout** is a regularization technique used to prevent overfitting in neural networks.
- ▶ During training, it randomly drops (set to 0) a fraction of neurons in each layer based on a specified probability for every forward pass.



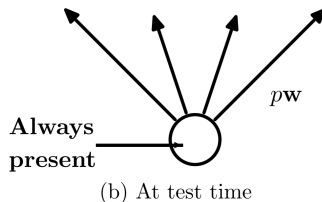
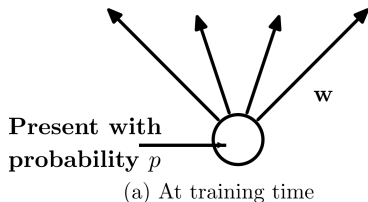
- ▶ Why is dropping neurons randomly useful?
 1. By dropping a subset of neurons during each forward pass, the network learns **not to rely too heavily on specific connections**, encouraging the network to use more connections.
 2. Dropout acts as an **efficient ensemble** of multiple smaller networks, each trained on a random subset of neurons.
 3. More Connections + Diverse Ensemble = **less overfitting**.



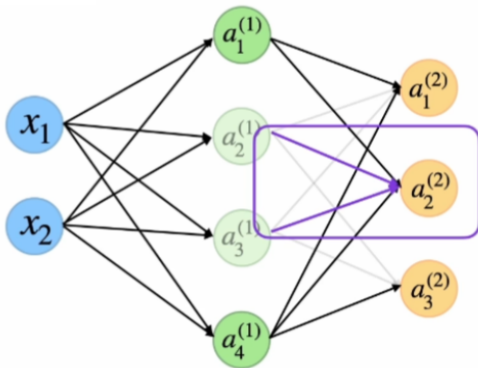
Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SVM on Fisher Vectors of Dense SIFT and Color Statistics	-	-	27.3
Avg of classifiers over FVs of SIFT, LBP, GIST and CSIFT	-	-	26.2
Conv Net + dropout (Krizhevsky et al., 2012)	40.7	18.2	-
Avg of 5 Conv Nets + dropout (Krizhevsky et al., 2012)	38.1	16.4	16.4

Table 6: Results on the ILSVRC-2012 validation/test set.

- ▶ We drop neurons during training, but what should we do during inference?
- ▶ **During inference:** We use all neurons.



- ▶ but...



Assume we have
present probability
 $p = 0.5$

During testing,
**when nodes are
present again,**
activations are
 $2 \times$ as large!

- ▶ **Problem:** Having inconsistent values during inference compared to training can cause the network to produce worse results.
- ▶ **Solution:** Scale the output during inference by p to keep the same expected activation as in training.
- ▶ **Example:**
 - **During training ($p=0.1$):**
 - ▶ Activation value = 2
 - ▶ Active only 10% of the time
 - ▶ $\Rightarrow$ Effective contribution = $2 \times 0.1 = 0.2$
 - **During inference:**
 - ▶ Always active ($\Rightarrow$ Activation value = 2)
 - ▶ Scale by (p) = 0.1

$2 \times 0.1 = 0.2$
 - **Result:** Consistent activation scale between training and inference.

- ▶ In PyTorch, dropout behavior is controlled by the model's mode:

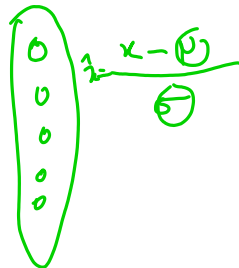
```
# Training Mode: Enables dropout
model.train()
output = model(input)
# Evaluation Mode: Disables dropout, scales activations
model.eval()
output = model(input)
```

- ▶ **Key Insight:** Networks train better when inputs are normalized
- ▶ So why not normalize intermediate layers too?
- ▶ **Problem:** forcing every layer to output zero-mean, unit-variance values might be too restrictive (not always optimal).
- ▶ **Solution:** Let the network learn if it wants normalization!

For each feature in a layer:

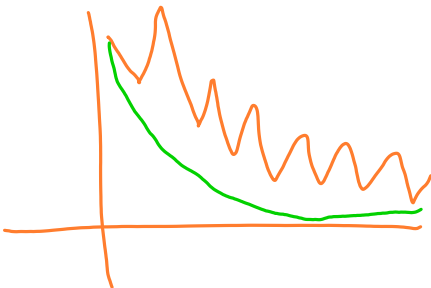
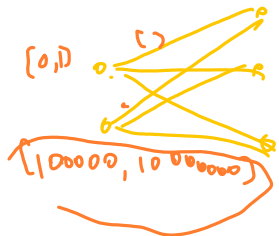
1. First normalize: $\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$

- μ : mean across batch dimension
- σ^2 : variance across batch dimension
- Computed separately for each feature/channel



2. Then give the network control: $y = \gamma \hat{x} + \beta$

- γ (scale): Can amplify or reduce the normalized values
- β (shift): Can move the values away from zero
- These are learned during training like normal weights!



- ▶ After normalization, outputs are always zero-mean, unit-variance
- ▶ But this might not be optimal for every layer!
- ▶ γ and β let each layer learn:
 - γ : "How much variance do I want?"
 - β : "What should my mean activation be?"
- ▶ Two extremes the network can learn:
 - "Keep normalization": $\gamma \approx 1, \beta \approx 0$
 - "Undo normalization": γ and β restore original scale and shift
- ▶ Network learns what's best for each layer!

Mathematically, for batch size N , at each feature/channel j :

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad (\text{batch mean})$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad (\text{batch variance})$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad (\text{normalize})$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad (\text{learnable transform})$$

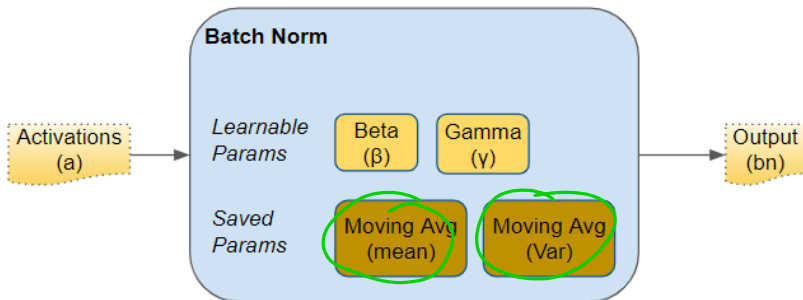
► During Training:

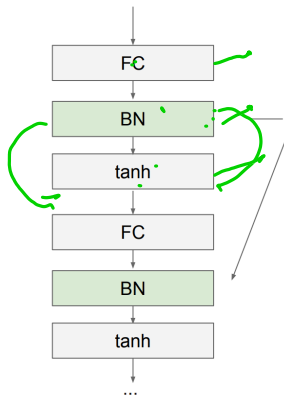
- Use statistics from current batch
- Keep running average of mean and variance (to use later in inference):

$$\mu_{running} = 0.9 \times \mu_{running} + 0.1 \times \mu_{batch}$$

► During Inference (We can't depend on mini-batches):

- Use stored running averages ($\mu_{running}, \sigma_{running}^2$)
- These are fixed values from training
- No batch statistics needed!





Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

- In Pytorch, batch normalization behavior is controlled by model's mode:

```
# Training Mode: Use batch statistics
model.train()
output = model(input)
# Evaluation Mode: Use running statistics
model.eval()
output = model(input)
```

▶

- Makes deep networks much easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!

Navigation icons: back, forward, search, and other controls.

► Advantages:

- Makes deep networks much easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!

► Disadvantages:

- Behaves differently during training and testing: this is a very common source of bugs!

⁰Slide based on CS231n by Fei-Fei Li, Yunzhu Li & Ruohan Gao

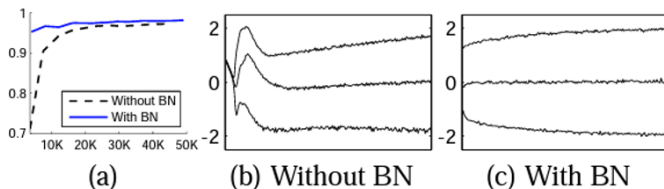


Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as $\{15, 50, 85\}$ th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*